

Reflection Reflection

Jania Vandevoorde



Overview

- Reflection, defined
- Origins (Smith 1984, Maes 1987, Cointe 1987)
- Applications (Chiba 1995, Chiba 2000)
- Optimizations (Marr et al. 2015)

What is Reflection?

- Programs treated as data within themselves
- Enables *metaprogramming*

Structural reflection

Python: **`isinstance()`**
C: **`sizeof()`**

Behavioral reflection

Changing a class's definition,
methods, access modifiers



Behavioral Reflection

```
1 import java.lang.reflect.*;
2
3 class Person {
4     private String name = "Alice";
5
6     private void sayHello() {
7         System.out.println("Hello from " + name);
8     }
9 }
10
11 public class Main {
12     public static void main(String[] args) throws Exception {
13         Person p = new Person();
14
15         Field field = Person.class.getDeclaredField("name");
16         field.setAccessible(true);
17         field.set(p, "Bob");
18
19         Method method = Person.class.getDeclaredMethod("sayHello");
20         method.setAccessible(true);
21
22         method.invoke(p);
23     }
24 }
```

Output

Hello from Bob

Why?

- Debugging systems, error handlers, and trace packages
- Dynamic code optimizers and macros
- Marshall/unmarshall data in different formats

Reflection and Semantics in Lisp

Brian Cantwell Smith

XEROX Palo Alto Research Center
3333 Coyote Hill Road, Palo Alto, CA 94304; and
Center for the Study of Language and Information
Stanford University, Stanford, CA 94305

1. Introduction

For three reasons, Lisp's self-referential properties have not led to a general understanding of what it is for a computational system to reason, in substantial ways, about its own operations and structures. First, there is more to reasoning than reference; one also needs a theory, in terms of which to make sense of the

and continuation characterising the state of the computation at that point. Thus, such constructs as `THROW` and `CATCH`, which must otherwise be provided primitively, can in 3-Lisp be easily defined as user procedures (and defined, furthermore, in code that is almost isomorphic to the λ -calculus equations one normally writes, in the metalanguage, to describe them). And all this can be done without writing the entire program in a

Reflective Systems

For a program to reason about itself, it needs

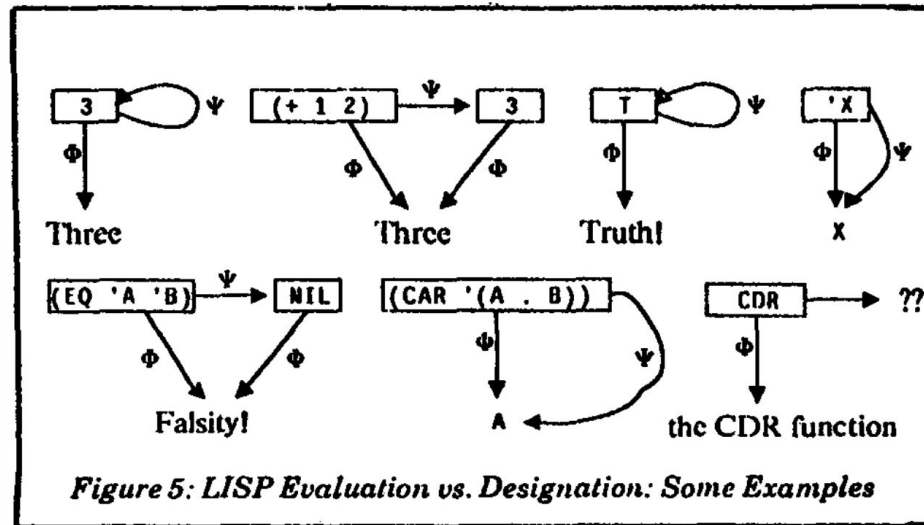
1. **A theory of itself:** a model of how it works embedded within it
2. **A live connection:** the model stays synchronized with what the program is doing
3. **The right distance:** close enough to see details, far enough to avoid tripping over itself

The issue with existing Lisp: `(+ 1 2)` and `(+ 1 (+ 1 1))` both get simplified to **3**

Computational Semantics

Designation Φ : what a structure refers to in the world

Normalization Ψ : what a the processor does with a structure



2-Lisp

Keep Φ and Ψ strictly separate

“A semantically rationalized dialect”

‘X designates the structure of X

↑ X ↓ ‘X ↓ ‘X ↓ X

Metacircular interpreters (MCP)

Lisp interpreter written in Lisp

Continuations (+ 1 (* 2 3))

<u>Type</u>	<u>Designation</u>	<u>Normal</u>	<u>Canonical</u>		<u>Notation</u>
Numerals	Numbers	Yes	Yes	—	<i>digits</i>
Booleans	Truth-Values	Yes	Yes	—	<i>\$T or \$F</i>
Handles	Structures	Yes	Yes	—	<i>'STRUC</i>
Closures	Functions	Yes	No	CCONS	<i>(closure)</i>
Rails	Sequences	Some	No	RCONS	<i>[STRUC ... STRUC]</i>
Atoms	(Φ of Binding)	No	—	ACONS	<i>alphanumeric</i>
Pairs	(Value of App.)	No	—	PCONS	<i>(STRUC . STRUC)</i>

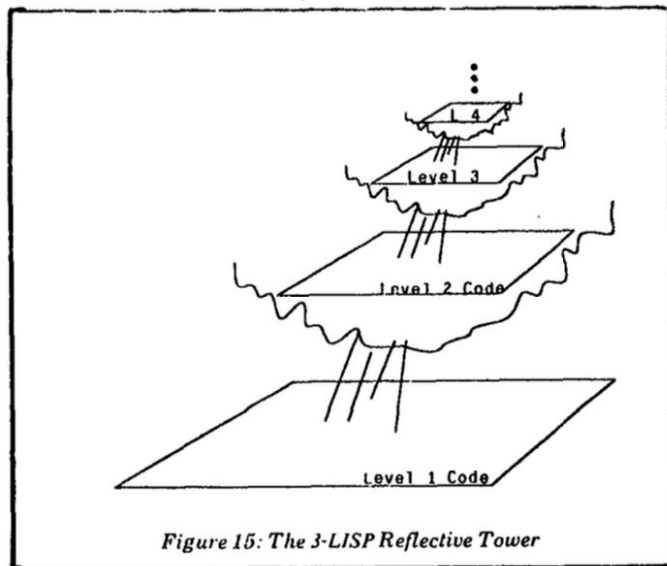
Table 1: The 2-LISP (and 3-LISP) Categories

3-Lisp & the Reflective Tower

3-Lisp = 2-Lisp + the *reflective tower*

- When *reflective procedures* are invoked, they're run with a MCP at the level above
- Finitely implemented

```
(lambda reflect [args env cont] ...)
```



The Reflective MCP

```

1 ..... (define READ-NORMALISE-PRINT
2 ..... (lambda simple [level env stream]
3 ..... (normalise (prompt&read level stream) env
4 ..... (lambda simple [result] ; Continuation C-REPLY
5 ..... (block (prompt&reply result level stream)
6 ..... (read-normalise-print level env stream))))))

[define REDUCE
... (lambda simple [proc args env cont]
..... (normalise proc env
..... (lambda simple [procl] ; Continuation C-PROC!
..... (if (reflective procl)
..... (↓(de-reflect procl) args env cont)
..... (normalise args env
..... (lambda simple [args!] ; Continuation C-ARGS!
..... (if (primitive procl)
..... (rest +1 args)
..... (define NORMALISE-RAIL
26 ..... (lambda simple [rail env cont]
27 ..... (if (empty rail)
28 ..... (cont (rcons))
29 ..... (normalise (1st rail) env
30 ..... (lambda simple [first!] ; Continuation C-FIRST!
31 ..... (normalise-rail (rest rail) env
32 ..... (lambda simple [rest!] ; Continuation C-REST!
33 ..... (cont (prep first! rest!))))))
34 ..... (cont (prep first! rest!))))))

```

Figure 17: The 3-Lisp Reflective Processor:

3-Lisp: a Reflective System

For a program to reason about itself, it needs

1. A theory of itself - handles & the reflective MCP
2. A live connection - reflective procedures get **env** and **cont**
3. The right distance - run one level up

Isn't it infinite?

- MCP runs programs tail-recursively
- MCP itself is tail-recursive

Significance

- Principled account of reflection rather than ad hoc tricks
- Continuations and environments can be first class objects
- Argues reflection should be treated as fundamental as recursion

Evaluation & Future Work

- No formal semantics defined for 3-Lisp
- Closures unnecessarily leak information
- Only structural reflection, not behavioral
- No reflective λ calculus or mathematical theory
- Want the option to specify which parts we want to be reflective

**CONCEPTS AND EXPERIMENTS IN
COMPUTATIONAL REFLECTION**

Pattie Maes

AI-LAB

Vrije Universiteit Brussel

Pleinlaan 2

B-1050 Brussels

pattie@arti.vub.uucp

ABSTRACT. This paper brings some perspective to various concepts in computational reflection. A definition of computational reflection is presented, the importance of computational reflection is discussed and the architecture of languages that support reflection is studied. Further, this paper presents a survey of some experiments in reflection which have been performed. Examples of existing procedural logic-based and

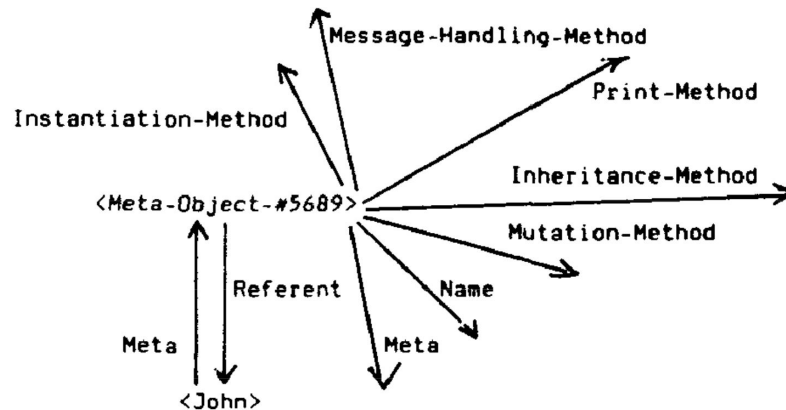
Reflective architectures have already been realised for procedure-based (Smith,1982), logic-based (Weyhrauch, 1980) and rule-based languages (Davis,1982). The second part of the paper (sections 6 to 8) discusses the realisation of a reflective architecture in an object-oriented language (further on called OOL). Existing OOLs only support limited, ad-hoc reflective facilities, which leads to limitations and unclear

3-KRS (Knowledge Representation System)

Every object has a corresponding metaobject strictly separated from itself

MCP + another interpreter to avoid infinite regress

Reflect on instances, classes, stack frames, and other OO constructs



Evaluation & Future Work

Illustrative evaluation

Implementation is not trivial

Proof of concept - leaves some questions open

Metaclasses are First Class : the ObjVlisp Model

Pierre Cointe
Rank Xerox & LITP

RXF: DRBI, 12 Place de l'Iris - Cedex 38, 92071 La défense
LITP: Université Paris-6, 4 place Jussieu, Tour 55-65, 75223 Paris
Email: ...!seismo!inria!litp!pc cointe@inria.inria.fr.uucp

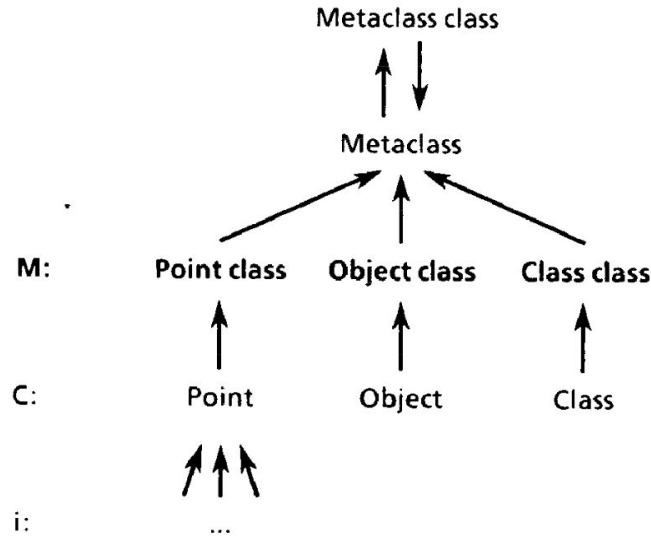
Abstract

This paper shows how an attempt at a uniform and reflective definition resulted in an open-ended system supporting ObjVlisp, which we use to simulate object-oriented language extensions.

We propose to unify Smalltalk classes and their terminal

tors, using a copy mechanism [9] or designing a delegation mechanism [15] are other themes developed by O.O programming.

Cointe and Maes



Because every metaclass is automatically an instance of the **Metaclass** class, metaclasses are not true classes, the number of metalevels is fixed and the metalink cannot be created indefinitely [17].

ObjVlisp

Single object type - no structural distinction between class, metaclass, and instance

name: class name

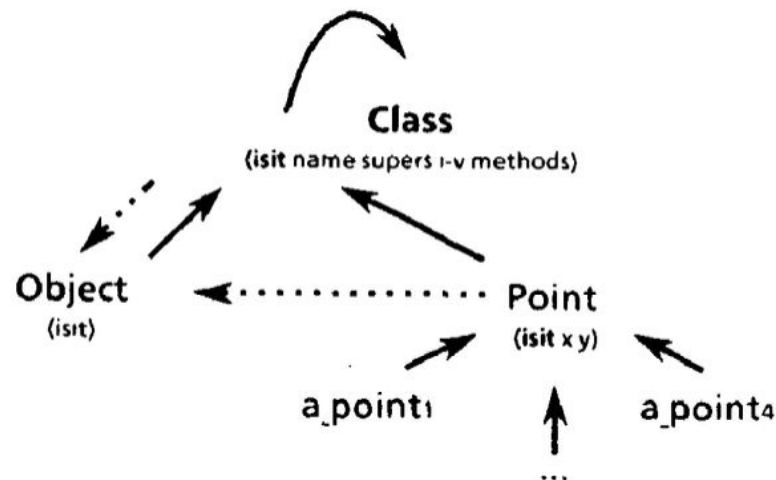
supers: direct superclasses

i_v: instance vars the class specifies

methods: method dictionary (name, λ)

Circular bootstrap to avoid infinite regress

Parameterized class support



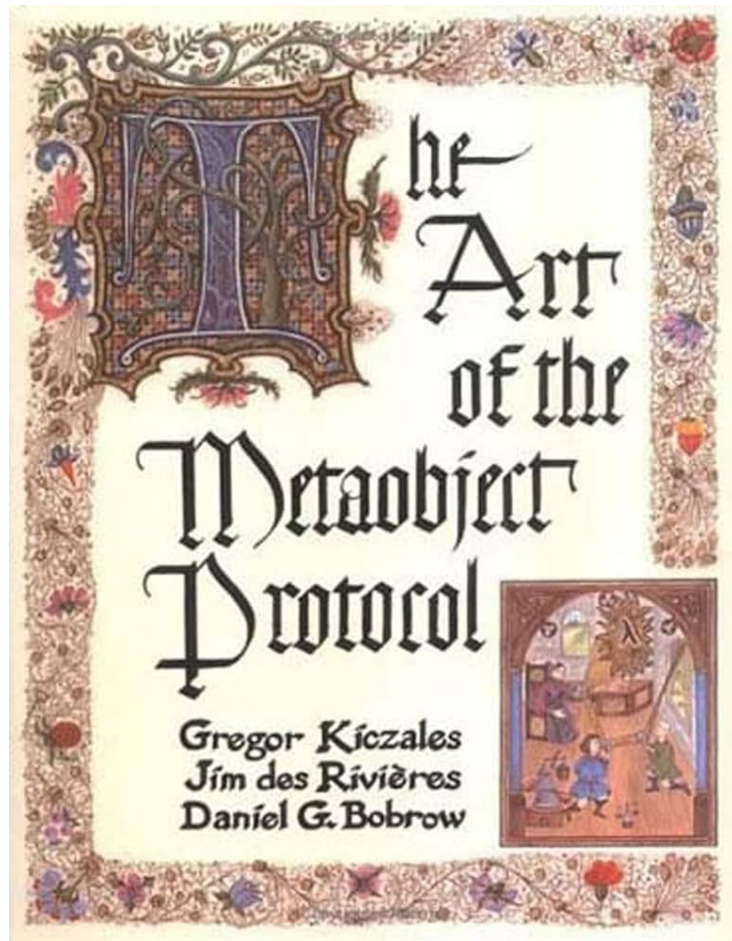
Evaluation & Future Work

Illustrative evaluation

Multiple inheritance of metaclasses has consequences

Reduces the separation between levels

Borning: metaclasses are already a difficult concept for Smalltalk learners



A Metaobject Protocol for C++

Shigeru Chiba

Xerox PARC & University of Tokyo

chiba@parc.xerox.com

chiba@is.s.u-tokyo.ac.jp

Abstract

This paper presents a metaobject protocol (*MOP*) for C++. This MOP was designed to bring the power of meta-programming to C++ programmers. It avoids penalties on runtime performance

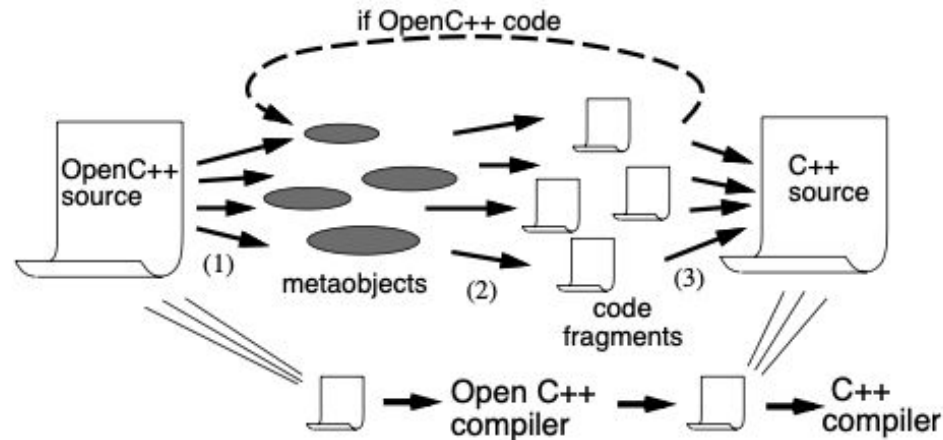
as inlining of matrix arithmetic. These can be implemented as libraries¹ and then used repeatedly. Unlike previous MOPs, our proposal incurs *zero* runtime speed or space overhead compared to ordinary C++.

Open C++

Metaobjects only exist before compile time

Metaclass annotation (**metaclass Node : PersistentClass**)

Silently rewrites member accesses, object creations, and function calls



Evaluation & Future Work

Illustrative evaluation

Tradeoff: expressiveness vs. runtime overhead

Introduces compile-time overhead

Doesn't address multiple metaclass inheritance

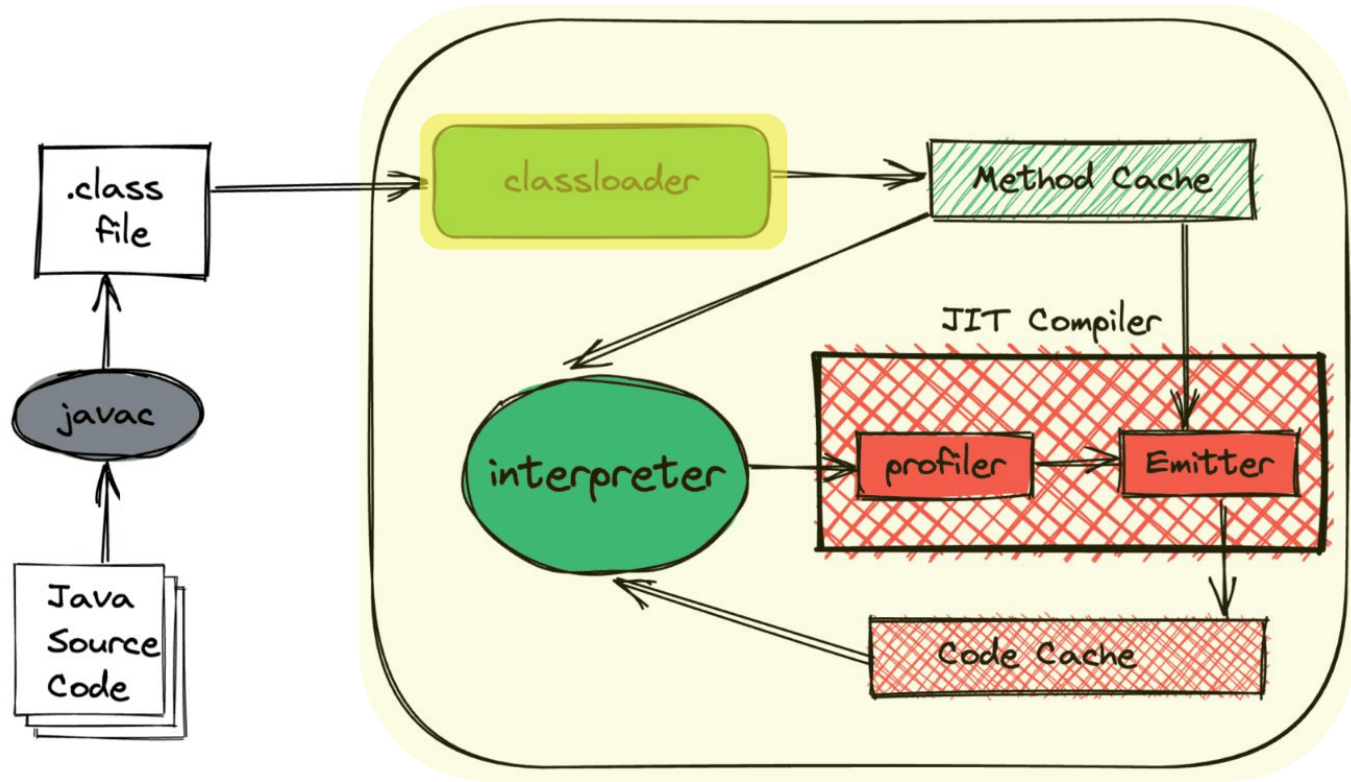
Load-Time Structural Reflection in Java

Shigeru Chiba

Institute of Information Science and Electronics
University of Tsukuba
and Japan Science and Technology Corp.
`chiba@is.tsukuba.ac.jp`, `chiba@acm.org`

Abstract. The standard reflection API of Java provides the ability to introspect a program but not to alter program behavior. This paper presents an extension to the reflection API for addressing this limitation. Unlike other extensions enabling behavioral reflection, our extension cal-

JVM



class file creation

Just In time (JIT) compilation

Load-Time Structural Reflection in Java

Javassist

Structural reflection at load time

CtClass - compile-time class

Table 3. Methods for alteration

Method in CtClass	Description
<code>void bePublic()</code>	make the class public
<code>void beAbstract()</code>	make the class abstract
<code>void notFinal()</code>	remove the final modifier from the class
<code>void setName(String name)</code>	change the class name
<code>void setSuperclass(CtClass c)</code>	change the super class
<code>void setInterfaces(CtClass[] i)</code>	change the interfaces
<code>void addConstructor(...)</code>	add a new constructor
<code>void addDefaultConstructor()</code>	add the default constructor
<code>void addAbstractMethod(...)</code>	add a new abstract method
<code>void addMethod(...)</code>	add a new method
<code>void addWrapper(...)</code>	add a new wrapped method
<code>void addField(...)</code>	add a new field
Method in CtField	Description
<code>void bePublic()</code>	make the field public
Method in CtMethod	Description
<code>void bePublic()</code>	make the method public
<code>void instrument(...)</code>	modify a method body
<code>void setBody(...)</code>	substitute a method body
<code>void setWrapper(...)</code>	substitute a method body

Javassist

When should we do reflection?

-  Compile time - need source code to transform classes
-  Runtime - lose type safety and JVM optimizations
-  Load time - direct byte code substitution where necessary

Evaluation & Future Work

Illustrative evaluation

Applications more naturally expressed than before

Benchmarked against OpenJava (10-13x faster)

- Skips source code parsing and recompilation - just manipulates bytecode

Expressiveness limited - no truly dynamic runtime reflection

Mirrors

Mirrors: Design Principles for Meta-level Facilities of Object-Oriented Programming Languages

Gilad Bracha
Sun Microsystems
4140 Network Circle
Santa Clara, CA 95054
(408) 276-7025
gilad.bracha@sun.com

David Ungar
Sun Microsystems
2600 Casey Ave., MTV 29-XXX
Mountain View, CA 94043
(650) 336-2618
david.ungar@sun.com

ABSTRACT

We identify three design principles for reflection and metaprogramming facilities in object oriented programming languages. *Encapsulation*: meta-level facilities must encapsulate their implementation. *Stratification*: meta-level facilities must be separated from base-level functionality. *Ontological correspondence*: the ontology of meta-level facilities should

```
Car anotherCar = theCarsClass.newInstance();
```

```
Class theCarsSuperclass = theCarsClass.getSuperclass();
```

Looking at the APIs of such a system, we expect to see something like:

```
class Object {  
  Class getClass();
```

Zero-Overhead Metaprogramming

Reflection and Metaobject Protocols

Fast and without Compromises



Stefan Marr

RMoD, Inria, Lille
France

stefan.marr@inria.fr

Chris Seaton

Oracle Labs / University of Manchester
United Kingdom

chris.seaton@oracle.com

Stéphane Ducasse

RMoD, Inria, Lille
France

stephane.ducasse@inria.fr

Abstract

Runtime metaprogramming enables many useful applications and is often a convenient solution to solve problems in a generic way, which makes it widely used in frameworks, middleware, and

Ruby, or Smalltalk, the runtime metaprogramming facilities enable DSLs designers to tailor the host language to enable more concise programs. Metaobject protocols (MOPs), as in Smalltalk or CLOS, go beyond more common metaprogramming techniques

Dispatch Chains

Cache metaobjects and methods for speedup at reflective call site

JIT compilation

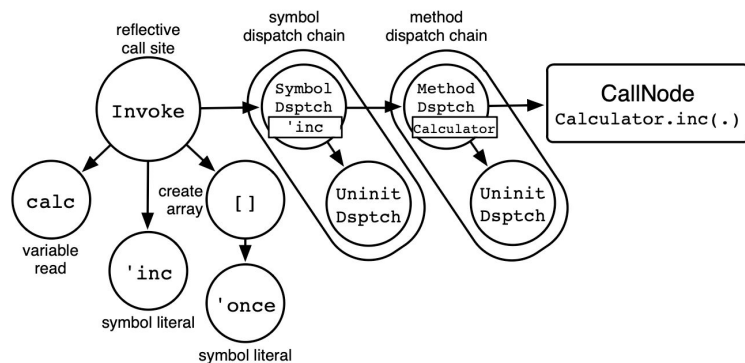


Figure 1. AST for: `calc.invoke('inc', ['once'])` with dispatch chain at the `invoke()` call site. The first level of the dispatch chain records the observed symbols for method names as indicated with the `'inc'` symbol as part of the cached symbol node. The second level then is a classic polymorphic inline cache for direct method dispatch, which caches the objects class (`Calculator`) for comparison and refers to the `CallNode` representing the actual method for direct invocation.

Evaluation & Future Work

10-20x speedup on psd.rb (Ruby image processing library)

Indirect invocations in SOM (minimal educational Smalltalk) have zero overhead

Unavoidable ~3-9% overhead from runtime correctness guards

Memory complexity not addressed

JIT compilation parameters may need tuning

Summary

1. Reflection and Semantics in Lisp (Smith, 1984)
2. Concepts and Experiments in Computational Reflection (Maes, 1987)
3. Metaclasses are First Class: the ObjVlisp Model (Cointe, 1987)
4. A Metaobject Protocol for C++ (Chiba, 1995)
5. Load-Time Structural Reflection in Java (Chiba, 2000)
6. Zero-Overhead Metaprogramming (Marr et al., 2015)

What's next?

C++ 26 native static reflection

Modern research mostly focused on reducing overhead

Complexity vs. expressiveness tradeoff

Questions
Questions

